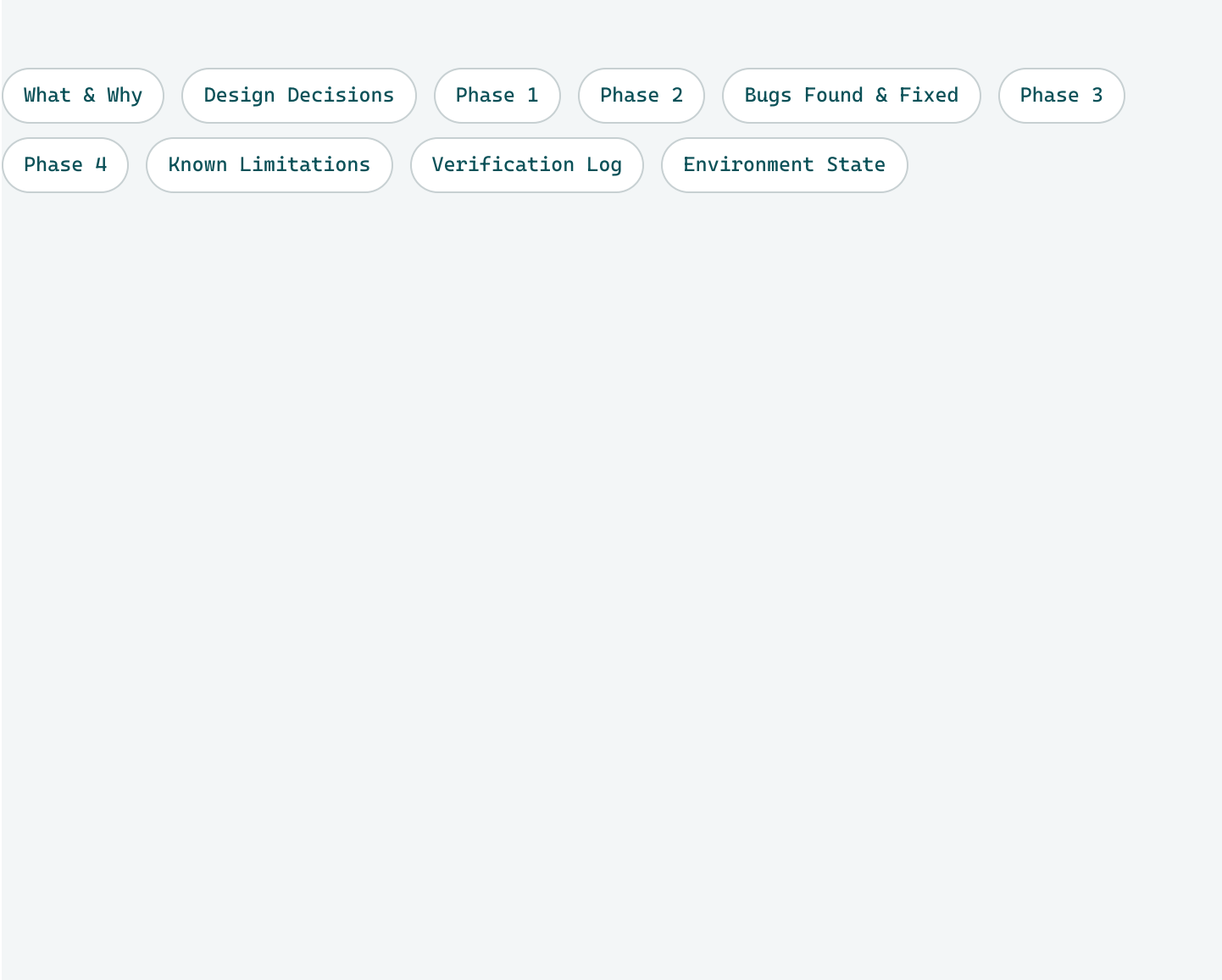


Verifier / Critic Agent

A standalone, deep-dive reference for Build 7 — a second, cheaper agent that checks the main agent's proposed answer against the actual tool evidence from that turn, before it's surfaced. Every phase, every real bug the process surfaced, and every verification result, in full. Complements (does not replace) the living cheat sheet and the per-phase flow diagrams.

STATUS	COMMITTS	CONTINUES
Complete — all 4 phases, live-verified in a real browser	88a5eb9 → 12acaed	Build 6 (front end for the agent)
PRECEDES		
None scoped yet — "Project 2" (a separate future portfolio project) is flagged but explicitly deferred		



What Was Built, and Why

Every eval case built since Build 4 checks the agent's behavior offline, against a fixed set of questions run by hand. None of that infrastructure does anything once the agent is actually answering a real question live. Build 7 closes that gap: a second, narrowly-scoped agent whose only job is judging whether the main agent's proposed answer actually follows from the tool evidence gathered for it — the eval harness's "verify against ground truth" ethos, extended into runtime.

Scoped originally during a 2026-07-24 placement/portfolio-strategy discussion (not a technical gap found hands-on, unlike every other build so far) — picked over a router/supervisor alternative specifically because that tool-choice judgment call was already handled reasonably well by the main agent's own system prompt (a Build 3 design point), so a router wouldn't add real value the way a verifier does.

Nothing about the main agent's own reasoning changed in this build. The verifier is a separate agent, called after the main agent already has its answer, reading the same structured tool-call log Build 4 already produces. No changes to `tools.py`, the tool schemas, or `SYSTEM_PROMPT`.

Design Decisions

Four forks resolved in a dedicated scoping conversation before any Phase 2+ code was written.

1. The Verifier's Own Model & Prompt

The original scoping note called it "a second, cheaper agent" — worth naming directly: the main agent already runs on `claude-haiku-4-5`, the cheapest/fastest current-gen Claude model, so there's no smaller tier to drop to. The real cost saving is **scope**, not model tier: the verifier's prompt is a single bounded judgment (question + tool evidence + proposed answer → supported/unsupported), never the full multi-turn conversation history the main agent carries. Same model, narrower prompt, and a forced `tool_choice` so the verdict is always a structured field, never prose that needs parsing.

2. Where the Verdict Surfaces

OPTION	VERDICT
Log only (Phase 2, built)	A new <code>log_verification()</code> event, same shape as <code>log_final_answer()</code> . Zero UX risk, proves the logic works before anything user-facing depends on it.
UI trust badge (Phase 3, built)	Non-blocking green/red status in the browser. Low risk since it only informs, never gates.
Blocking/retry a flagged answer	Deferred, not built. Highest complexity and real risk – a verifier false-positive would become a live UX regression, not just log noise. See Known Limitations.

3. Every Turn, or Opt-In?

Full always-on doubles the API call count per question. Pure opt-in feels bolted-on rather than integral. The chosen middle ground reuses a signal that already existed rather than adding new state: **skip verification when a turn made zero tool calls** — there's no evidence to check an answer against if none was gathered (e.g. "What can you do?"). `get_tool_calls(question_id, include_output=True)` returning an empty list is the gate.

4. Phase Breakdown

Mirrors this project's established shape for every prior build: prove the mechanism standalone first (Phase 1, same pattern as `search_summaries.py` predating `tools.py`'s wired version), wire it into the real system second (Phase 2), surface it to a user third (Phase 3), then a stretch phase proving the mechanism actually catches failures rather than assuming it does (Phase 4) — the same rigor Build 4's eval harness already established for the main agent, applied here to the verifier itself.

1 Standalone Verifier Proof

Goal: prove the verdict logic works in isolation, before touching `agent.py` — same standalone-first pattern as every prior build's Phase 1.

`verify_answer.py` — the core judgment call

```
VERIFIER_SYSTEM_PROMPT = """You check whether a proposed answer is actually supported by the tool
evidence gathered for it. You are not answering the original question yourself and must not use any
outside knowledge - judge only whether the proposed answer logically follows from the tool call
inputs/outputs shown to you.
```

```
Call report_verdict with supported=true only if every claim in the proposed answer is backed by the
tool evidence. If the answer contradicts the evidence, overstates it, or claims something the
evidence never shows, set supported=false and explain why in reason."""
```

```
def verify_answer(user_question, answer, tool_calls):
    evidence = build_evidence_block(tool_calls)
    response = client.messages.create(
        model="claude-haiku-4-5",
        system=VERIFIER_SYSTEM_PROMPT,
        tools=[VERIFIER_TOOL],
        tool_choice={"type": "tool", "name": "report_verdict"},
        messages=[{"role": "user", "content": user_content}],
    )
    for block in response.content:
        if block.type == "tool_use" and block.name == "report_verdict":
            return block.input["supported"], block.input["reason"]
```

The script's CLI accepts a real `question_id` from `logs/tool_calls.jsonl`, plus an optional second argument that overrides the logged answer — one script proves both the good case (real evidence + real answer) and a deliberately-bad case (real evidence + a hand-typed wrong answer) without needing two separate scripts.

Sample — live proof, both directions, same real tool evidence

```
# python verify_answer.py b524250c6b11
```

```
question: What type of gear do you have?
```

```
supported: True
```

```
reason: The proposed answer is fully supported by the tool evidence. The SQL query
returned 20 sample summaries...all the gear types and specific products mentioned
in the proposed answer...are directly present in the query results.
```

```
# python verify_answer.py b524250c6b11 "There is no camera equipment available in the inventory."
```

```
supported: False
```

reason: The proposed answer contradicts the tool evidence. The SQL query output clearly shows extensive camera equipment in the inventory, including: NIKON Z6 III MIRRORLESS BODY, OLYMPUS OM-D E-M1 BODY, FUJIFILM X-T5 BODY...

Both verdicts cite specific evidence (item names, actual query rows) in their reasoning, not generic filler — the thing that actually matters for Phase 4 to later prove wasn't a coincidence.

2 Wire the Verifier Into the Agent

Goal: run the verdict automatically on real traffic, gated on the zero-tool-call rule from the scoping decisions above, without ever risking the main answer.

agent.py – the whole integration, inside ask_agent()'s success return

```
if response.stop_reason != "tool_use":
    log_final_answer(question_id, user_question, answer)
    try:
        evidence = get_tool_calls(question_id, include_output=True)
        if evidence:
            supported, reason = verify_answer(user_question, answer, evidence)
            log_verification(question_id, user_question, answer, supported, reason)
    except Exception as e:
        print(f"[VERIFY ERROR] {e}")
    return {"answer": answer, "error": None, "full_messages": messages, "question_id": question_id}
```

get_tool_calls gained an include_output flag, default False. The Phase 4 (Build 6) UI caller never needed raw tool output — only the verifier does, since judging an answer requires seeing what the tools actually returned, not just that they ran. Defaulting False means the existing UI caller and its tests are entirely unaffected by this change.

log_verification()'s entry deliberately uses answer_text, not answer, as the key. get_final_answer() already filters on "answer" in entry to find log_final_answer()'s entries — reusing that exact key on a verification entry would create a fragile dependency on write order (the final-answer entry happening to be written first) rather than a structurally impossible collision. get_verification() filters on "supported", a key only its own entries ever carry.

Verified live against real traffic, both branches of the gate:

Sample – real question, real tool calls, real verifier call

Q: "What kind of camera equipment has been checked out recently?"

-> 3 tool calls (list_tables, search_summaries, run_sql_query), real answer

-> log entry: supported=True, reason cites specific allocation_ids and department codes from the actual SQL result set

Q: "What can you do?"

-> 0 tool calls, answered entirely from the system prompt

-> log entry: only the plain log_final_answer entry - no "supported" key anywhere, gate correctly skipped verification entirely

Bugs Found & Fixed

Two real, would-have-broken-production bugs, both caught by review before ever being run — a direct payoff of this project's standing "inspect the entry before testing" habit.

1. A one-line indentation error in agent.py that would have raised IndentationError on the very first import. The `if response.stop_reason != "tool_use":` line landed at 12 spaces instead of 8, one level deeper than the plain statement directly above it — Python requires the same indent level after a line that doesn't open a new block, so this wasn't just cosmetically wrong, it was a hard parse failure that would have stopped every question, not just verification.

Fixed: dedented the single line back to 8 spaces. Confirmed via a precise per-line indentation dump (not eyeballing) that every line below it was already correctly nested relative to that fix — one line was the entire bug, not a cascade.

2. The Dockerfile's COPY line never picked up verify_answer.py — same bug class as Build 4's logging_utils.py omission in Build 6. agent.py now does `from verify_answer import verify_answer`, but the image would have built "successfully" and then failed with `ImportError` the moment the container actually started, since the file was never copied in at all.

Fixed: `COPY agent.py tools.py logging_utils.py app.py verify_answer.py ./`. Verified directly inside the running container (not assumed from the build log) via `docker exec ... ls //app/verify_answer.py` and a `grep` confirming the container's own copy of agent.py/app.py actually has the new imports and calls.

3 UI Trust Badge

Goal: surface the verdict to a human, non-blocking, matching the "log only, then UI flag" phased rollout from the design decisions above.

```
app.py - the whole rendering logic

def render_verification_badge(verification):
    if verification is None:
        return
    if verification["supported"]:
        st.success("✓ Verified against tool evidence")
    else:
        st.error(f"⚠ Unverified: {verification['reason']}")
```

Streamlit built-ins over raw HTML, for a real reason, not just convenience. A colored `<span style='color:... '>` via `unsafe_allow_html=True` was considered for exact color control — rejected because the reason text embedded inside it is LLM-generated, not trusted input, and rendering it as raw HTML would be a real (if low-probability) XSS surface if the model ever produced text that looked like a tag. `st.success()`/`st.error()` give the same green/red semantic with Streamlit's own safe text escaping and no new attack surface.

verification is fetched via the same log-as-source-of-truth pattern `get_tool_calls/get_final_answer` already established, threaded into `round["verification"]` for both freshly-asked and persisted-history rounds, and rendered right before the existing Tools Used expander in all three places an answer appears on screen.

Live-confirmed in the browser: a real question through the fully rebuilt container correctly showed the green badge. A deliberately adversarial question ("What's the total dollar value of all Nikon Z6 III equipment currently checked out?" — `clean.allocations` has no price column at all) produced a well-behaved answer: the agent correctly found zero active checkouts and correctly declined to invent a dollar figure, which the verifier correctly marked `supported: true` — a good outcome, not a missed catch, since nothing in the answer actually overreached.

4 Verifier Eval Harness (Stretch)

Goal: prove the verifier actually catches a bad answer, not just that it never complains — the same bar Build 4's eval harness set for the main agent, applied here to the thing doing the checking.

Hand-constructed evidence, not the live log. Reading real historical `question_ids` would make the cases non-reproducible and CI-unfriendly — the exact lesson Build 5's `ci_skip` fix already learned the hard way about depending on real historical state. All three cases use `clean.allocations`, the agent's actual in-scope table — an earlier draft referenced `customers`, caught and corrected mid-session since that table was removed from `list_tables()/describe_table()` discovery back in Build 6, meaning the live agent can no longer naturally produce a scenario involving it.

```
run_verify_eval.py - mirrors run_eval.py's existing shape

for case in VERIFY_EVAL_CASES:
    supported, reason = verify_answer(case["user_question"], case["answer"], case["tool_calls"])
    passed = supported == case["expected_supported"]
    # full per-case report regardless of pass/fail, same as run_eval.py
if failures:
    sys.exit(1) # the same fix run_eval.py itself needed in Build 5
```

Real output — both deliberately-bad cases caught, not rubber-stamped

```
[PASS] grounded answer matches real evidence
expected_supported=True actual_supported=True
```

```
[PASS] answer contradicts the evidence outright
expected_supported=False actual_supported=False
reason: The proposed answer directly contradicts the tool evidence. The SQL query
returned two camera products...which clearly demonstrates that camera equipment
does exist in the inventory.
```

```
[PASS] answer invents a number the evidence never returned
expected_supported=False actual_supported=False
reason: There is no pricing information, dollar values, or totals shown in the
output...This answer appears to be unsupported speculation.
```

3/3 cases passed

The zero-tool-call skip path got a real pytest test instead of a costed API eval case. Whether `get_verification()` correctly returns `None` when no matching entry exists is pure Python logic — no reason to

spend an API call proving it. `test_verification.py` covers both the found and not-found branches via a temp log file + monkeypatch, the same pattern `test_chat_rounds.py` already established.

Known Limitations & Open Items

Blocking/retry behavior was scoped and explicitly deferred, not built. If a real answer gets flagged supported: `false` today, nothing happens beyond the log entry and the red badge — the main answer is never retried, hedged, or withheld. This was a deliberate Phase 2 design-decision boundary (see Design Decisions above), not an oversight: a verifier false-positive blocking a correct answer would be a real UX regression, and that risk shouldn't be taken on until the log-only and UI-flag phases have proven the verifier trustworthy over real usage.

The red/unsupported badge was proven at the data and component layer, not forced to render live in the browser. `get_verification()` returning the right shape for a known-bad case, and `st.error()` being a standard well-tested Streamlit built-in (same reliability class as `st.success()`, which did render correctly live) together give strong confidence without needing to hand-craft a fake `chat_rounds/log_verification` pair just to force the browser render — judged not worth the more invasive test-fixture setup for a UI branch that's structurally identical to the already-confirmed green one.

Process note, kept honest rather than smoothed over: the diagram-first ordering lapsed for Phase 1. Every prior build in this project publishes a phase's flow diagram before writing its code, confirmed by the git log itself. Phase 1's diagram was built retroactively, after the code already existed and worked — caught by direct question, not self-caught. Phases 2 through 4 restored the correct order, including the full dual-verification pass (`--screenshot` and `--print-to-pdf+pdftoppm`) that Phase 1's diagram also initially skipped and had to be re-run for.

Verification Log

What was actually run against a real environment, a real browser, or the real Anthropic API — not just read for plausibility.

Phase 1

Live-run twice against a real logged `question_id` (b524250c6b11) — once with the real logged answer (`supported: True`), once with a hand-typed wrong override answer (`supported: False`), both reasoning traces citing specific real evidence.

Phase 2

`py_compile` clean on all four touched files after the indentation fix; a real `ask_agent()` call with genuine tool calls confirmed a `supported: true` log entry with grounded reasoning; a second real call with zero tool calls confirmed no verification entry was written at all; full pytest suite 25/25 (unchanged — no regressions).

Phase 3

`py_compile` clean; `get_verification()` functionally confirmed against both real Phase 2 `question_ids` from a script; container rebuilt and recreated (catching the Dockerfile bug along the way), confirmed via `docker exec ... grep` that the running image actually has the new imports/calls; live-confirmed in the browser — a real question produced the correct green `st.success()` badge.

Phase 4

`run_verify_eval.py` — 3/3 cases passed, including both deliberately-bad cases correctly flagged `supported: False` with evidence-citing reasoning, not just a lucky guess; full pytest suite 27/27 (up from 25, the two new `get_verification()` tests included).

Environment State

COMPONENT	DETAIL
New Python	<code>verify_answer.py</code> (the verifier itself, importable + CLI-runnable), <code>verify_eval_cases.py</code> , <code>run_verify_eval.py</code> , <code>test_verification.py</code>
Changed Python	<code>agent.py</code> (<code>verify_answer</code> import, <code>verify-and-log</code> block in <code>ask_agent()</code> 's success return, wrapped in <code>try/except</code>), <code>logging_utils.py</code> (<code>get_tool_calls()</code> gained <code>include_output</code> , new <code>get_verification()/log_verification()</code>), <code>app.py</code> (<code>get_verification</code> import, <code>render_verification_badge()</code> , <code>verification</code> threaded into both persisted and new rounds)
Changed infra	<code>Dockerfile</code> (<code>COPY</code> line fixed to include <code>verify_answer.py</code>)
New tests	<code>test_verification.py</code> – 2 tests (<code>get_verification</code> found/not-found, temp log + monkeypatch, no API cost)
Diagram folder	<code>PROJECT_DIAGRAMS/BUILD_7_FLOW/</code> – phase1 through phase4 flow diagrams, this brief
Live app	<code>http://localhost:8501</code> (<code>streamlit_app</code> service) – rebuilt and recreated to pick up all four phases
Project test suite	27/27 passing (up from 25 at the start of this build)

Build 7 complete — all four phases, live-verified in a real browser and against the real Anthropic API. Nothing else currently scoped in the original five-build-plus roadmap; "Project 2" (a separate, standalone future portfolio project synthesizing a verify-before-act theme across structured and unstructured data) is flagged in memory but explicitly deferred, not started. Ongoing cross-build command reference lives in `sql-test-01-cheatsheet.html`.